CS/CE 224/227: Object Oriented Programming and Design Methodologies

Week 03-04/Unit 03: Pointers

Course taught by: Zubair Irshad

Slide courtesy: Faisal Alvi

(For any suggested modifications please email: faisal.alvi@sse.habib.edu.pk)

Logistics

1. Monday's missed class will be recorded and shared on Canvas.
    1. The plan is to do it for after Wednesday's class i.e. on Thursday so everyone has the weekend to watch the lecture. **We will do a short quiz next Monday to mark attendance for Monday's missed lecture**
2. Quiz 1 postponed to next Monday i.e. 15th September (**Prepare first two slide-decks**)

Unit Outline

1. Review of Last Week's Content: C-Strings
2. Pointers
3. Pointers and Functions
4. Pointers and Arrays
5. Arrays, Pointers and Functions
6. Summary

Arrays of Strings

```
#include <iostream>
using namespace std;

int main()
   {
   const int DAYS = 7;              //number of strings in array
   const int MAX = 10;              //maximum size of each string
                                    //array of strings
   char star[DAYS][MAX] = { "Sunday", "Monday", "Tuesday",
                            "Wednesday", "Thursday",
                            "Friday", "Saturday"  };
   for(int j=0; j<DAYS; j++)        //display every string
      cout << star[j] << endl;
   return 0;
   }
```

- What does the storage of this 2D array look

Arrays of Strings

```
#include <iostream>
using namespace std;

int main()
   {
   const int DAYS = 7;              //number of strings in array
   const int MAX = 10;              //maximum size of each string
                                    //array of strings
   char star[DAYS][MAX] = { "Sunday", "Monday", "Tuesday",
                            "Wednesday", "Thursday",
                            "Friday", "Saturday"  };
   for(int j=0; j<DAYS; j++)        //display every string
      cout << star[j] << endl;
   return 0;
   }
```

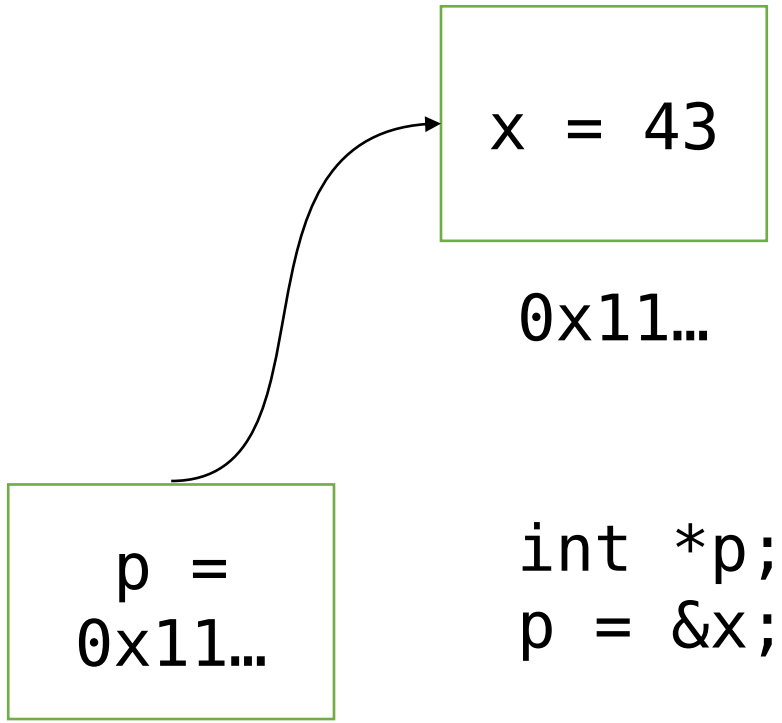
- What does the storage of this 2D array look

Row (Day)	Stored Characters (each column = 1 char, up to MAX=10)									
0 (Sunday)	S	u	n	d	a	y	\0	.	.	.
1 (Monday)	M	o	n	d	a	y	\0	.	.	.
2 (Tuesday)	T	u	e	s	d	a	y	\0	.	
3 (Wednesday)	W	e	d	n	e	s	d	a	y	\0
4 (Thursday)	T	h	u	r	s	d	a	y	\0	
5 (Friday)	F	r	i	d	a	y	\0	.	.	.
6 (Saturday)	S	a	t	u	r	d	a	y	\0	.

Each row corresponds to one **string** (day of the week).
Each column inside that row is a **character** in that string.

Pointers

- In C++, a pointer variable is an address to (or the memory location of) a variable.
- Let's say we define `int x = 43;`
- The variable `x` has a value of 43, and it has been allocated a memory location equal to 4 bytes.
- What is the actual memory address of x?
- This address can be founded (and printed) using a pointer variable.
- `int* p = &x;`



Pointers

Memory

0X1000	**43**
0X1004	
0X1008	
0X1012	
0X1016	
0X1020	

```
int x = 43
```

Pointers

Memory

0X1000	43
0X1004	**0X1000**
0X1008	
0X1012	
0X1016	
0X1020	

```
int x = 43

int* p = &x;
```

Pointers (Let's break down the syntax)

```
int x = 43
```

Integer named x is set to 43

```
int* p = &x;
```

Integer **Pointer** named **p** is set to **the address of** x

Pointers (Let's break down the syntax)

```
int* p = &x;
```

Integer **Pointer** named **p** is set to **the address of** x

Let's say we want to copy the value of x to a new variable using the pointer

```
int y = *p;
```

Integer named **y** is the **thing pointed to by** p

Pointers

- In C++, a pointer to a data type is declared as (type *).
- Therefore a pointer to an integer variable is declared as int*.
- Consider the following program →
- The output is:

```
x = 43
ptr = 0x61fe10
*ptr = 43
```

```cpp
#include <iostream>
using namespace std;

int main(void) {
    int x = 43, y;
    int* ptr;

    ptr = &x;
    y = *ptr;

    cout << "x = " << x << "\n";
    cout << "ptr = " << ptr << "\n";
    cout << "*ptr = " << y << "\n";

    return 0;
}
```

Referencing Operator

Dereferencing operator

Pointers

Memory

0X1000	43
0X1004	0X1000
0X1008	
0X1012	
0X1016	
0X1020	

```
int x = 43

int* p = &x;
```

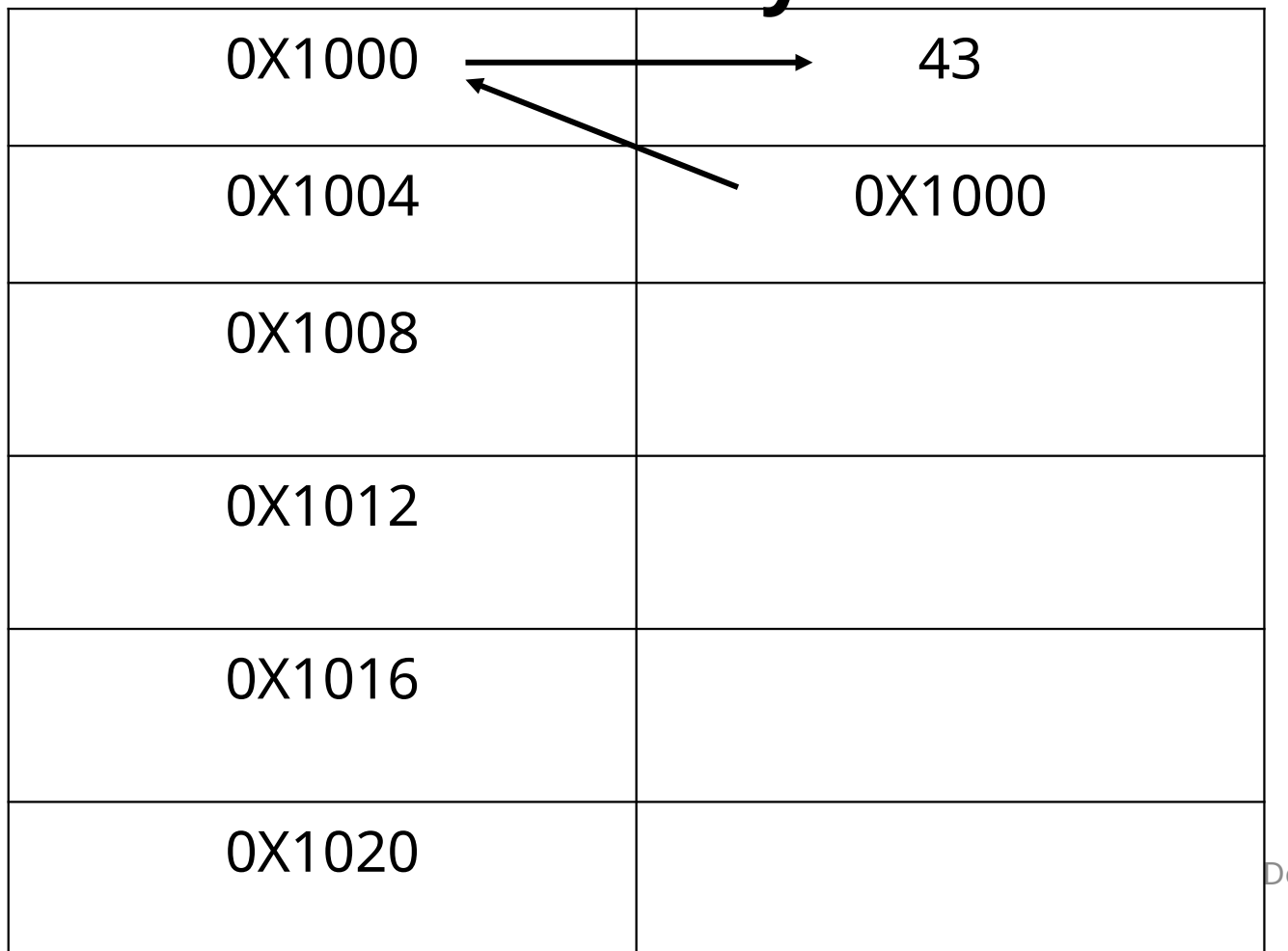
What is

int y = *p; ?

In other words, what is the thing pointed to by p?

Pointers

Memory




```
int x = 43

int* p = &x;
```

What is

```
int y  = *p; ?
```

In other words, what is the thing pointed to by p?

A. 43

Pointer Variables – Declaration

- The operator '&' is the referencing operator. It gets memory location of a variable
- The operator * is the dereferencing operator. It accesses the value at the stored address. **Note * should not be with a data type to act as dereferencing**
- Question: What is the output of: `int x = 4; cout << *(&x);`
- In C++, you need to declare the type of the variable the pointer is pointing to.
- So a pointer to a character is an address to a memory location having 1 byte, a pointer to an integer having 4 bytes and so on…
- In general, we can have `int* p, char* p, double* p` and so on.

- What is the answer of?

```
int z = 100;
int* p1 = &z;
int* p2 = p1;
cout << *p2;
```

- What is the answer of?

```
int z = 100;
int* p1 = &z;
int* p2 = p1;
cout << *p2;
```

A. 100

Pointers And Function

- Typically variables are passed by value in C++ in functions.
- What does pass by value mean – it means a *copy* of the variable is passed to the function.
- Any modification to the passed parameter does not change the value of the argument.
- Consider the following program →
- The objective is to transfer the amount in the source to the dest. It doesn't work!

```
#include <iostream>
using namespace std;
void transfer(int, int);

int main(void) {
    int source = 4, dest = 0;
    cout << "Before the transfer: ";
    cout << source << ", " << dest << endl;
    transfer(source, dest);
    cout << "After the transfer: ";
    cout << source << ", " << dest << endl;
    return 0;
}

void transfer(int source, int dest) {
    dest = dest + source;
    source = 0;
}

//OUTPUT IS
//Before the transfer: 4, 0
//After the transfer: 4, 0
```

Pass By Pointers

- With pointers you can pass a value using its memory address.
- Pass by pointers means that, instead of passing the value, you now pass the *address* of that value.
- Any changes to the value pointed to by the address changes the value of the argument too.
- Consider the following program now →.
- Observe the syntax.

```
#include <iostream>
using namespace std;
void transfer(int* , int* );

int main(void) {
    int source = 4, dest = 0;
    cout << "Before the transfer: ";
    cout << source << ", " << dest << endl;
    transfer(&source, &dest);
    cout << "After the transfer: ";
    cout << source << ", " << dest << endl;
    return 0;
}

void transfer(int* source, int* dest) {
    *dest = *dest + *source;
    *source = 0;
}

//OUTPUT IS
//Before the transfer: 4, 0
//After the transfer: 0, 4
```

Recap from last time, Coding Practice!

- “Write a program to swap two variables using pass by value, pass by pointer, and pass by reference.
  - Why does the swap succeed in some cases but fail in others?”

C++ Reference Variables

- A reference variable is an ***alias***, that is, another name for an already existing variable.
- Once a reference is initialized with a variable, either the variable name or the reference name may be used to refer to the variable.
- For example, consider

```
int a = 4;
```

```
int &b = a;
```

- b becomes a reference (alias) for a.
- Here b and a refer to the same variable and hence b is a reference to a. Any changes to b reflect in a also. **Changing a also changes b. Changing b also changes a. They are just two names for the same variable.**
- **Use case:** references are often used for **function parameters** (pass by reference) as a simpler alternative to pointers.
- **& in expression** → address-of operator. **& in declaration** → creates a reference (alias). Very Important!!

Differences between references and pointers

- You cannot have NULL references. You must always be able to assume that a reference is connected to a legitimate piece of storage.

```
int x = 10;
int &ref = x;  // valid
// int &ref2;  // ❌ not allowed, must initialize

int* p = nullptr;  // valid
```

Differences between references and pointers

- Once a reference is initialized to an object, it cannot be changed to refer to another object. Pointers can be pointed to another object at any time.

```
int a = 5, b = 10;
int &r = a;   // r refers to a
r = b;        // ❌ does NOT rebind r to b; it just assigns b's value to a
```

Differences between references and pointers

- A reference must be initialized when it is created. Pointers can be initialized at any time.

```
int x = 5;
int &r = x;    // ✅ ok
int &r2;       // ❌ not allowed


int x = 5;
int* p;    // declare
p = &x;    // assign later ✅
```

Pass By Reference

- Another method for passing parameters in C++ is pass by reference.
- With this, we create a reference to the parameter that is supposed to be passed.
- Consider the following program now →.
- Observe the syntax.
- Only need to pass by reference in **function definition**

```cpp
#include <iostream>
using namespace std;

// Function with pass by reference
void transfer(int& src, int& dst) {
    dst = dst + src;   // add source to destination
    src = 0;           // empty source
}

int main() {
    int source = 4, dest = 0;

    cout << "Before the transfer: ";
    cout << source << ", " << dest << endl;

    // Pass by reference call (no & or * needed here)
    transfer(source, dest);

    cout << "After the transfer: ";
    cout << source << ", " << dest << endl;

    return 0;
}

//OUTPUT IS
//Before the transfer: 4, 0
//After the transfer: 0, 4
```

Let’s look at all 3 together!

```cpp
#include <iostream>
using namespace std;

void transfer(int src, int dst) {   // copy of source, dest
    dst = dst + src;
    src = 0;
}

int main() {
    int source = 4, dest = 0;
    transfer(source, dest);         // normal call
    cout << source << ", " << dest; // Output: 4, 0
}
```

Pass by Value
(copy made, original unchanged)

```cpp
#include <iostream>
using namespace std;

void transfer(int* src, int* dst) { // pointers to source, dest
    *dst = *dst + *src;
    *src = 0;
}

int main() {
    int source = 4, dest = 0;
    transfer(&source, &dest);       // pass addresses
    cout << source << ", " << dest; // Output: 0, 4
}
```

Pass by Pointer
(addresses passed, dereference inside)

```cpp
#include <iostream>
using namespace std;

void transfer(int& src, int& dst) {
    // references to source, dest
    dst = dst + src;
    src = 0;
}

int main() {
    int source = 4, dest = 0;
    transfer(source, dest);// looks like pass by value
    cout << source << ", " << dest; // Output: 0, 4
}
```

Pass by Reference
(aliases created, cleaner syntax)

Pointers in Arrays

The name of the arrays points to the first memory location

Arrays and Pointers

- Recall that an array is a collection of homogeneous data types in C++.
- **The name of the array refers to the memory address of the first data item stored in the array.**
- a itself is the **address of the first element** (&a[0]).

a → **&a[0]**

- In this sense, the **name of the array is** by itself, **an address** and can be treated as a pointer.
- Try the following code snippet:

```
int a[] = {5, 3, 4, 2, 1};
cout << a <<endl;
for(int ix = 0; ix < 5; ix++){
cout << a + ix <<endl;
cout << *(a + ix) << endl;
}
```

```
#include <iostream>
using namespace std;


int main(){
    int a[] = {5, 3, 4, 2, 1};
    cout <<a; // prints the memory address of a[0]
}
```

Takeaway:
a[i] is equivalent to *(a + i).

Dynamic Memory Allocation

Static vs Dynamic Arrays

Static and Dynamic Arrays

- So far we have studied arrays that are initialized at compile time.
- For these arrays, the size has to be declared upfront.
- These automatically get deleted when the function defining them completes execution.
- These are known as static arrays and are allocated memory on the *stack*.
- Examples of static arrays:

```
    int a[5]; char name[80];
Or   int a[20][40]; char words[80][100]
```

Stacks vs Heap!

- Stack!
  - Memory is allocated in **contiguous blocks** within the **call stack**.
  - The size of memory required is **already known** before execution.
  - The programmer **does not** need to handle allocation or deallocation.

- Heap!
  - Heap memory is **not guaranteed** to be contiguous.
  - Heap allocation is typically used when the size is **not known until runtime**.
  - The programmer (or garbage collector, depending on the language) **must handle allocation/deallocation**.

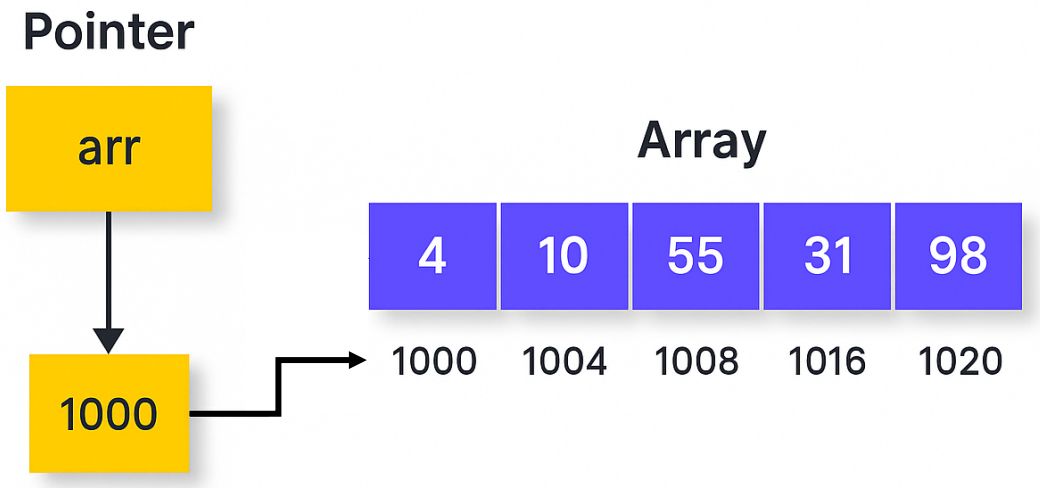
In short, static arrays → Stack, Dynamic Arrays → Heaps!

Dynamic Array in C++

- In C++, dynamic arrays allow users to allocate memory dynamically.
- They are useful when the size of the array is not known at compile time.
- In C++, a dynamic array can be created using new keyword and can be deleted by using `delete` keyword.
- How to initialize a dynamic array?

```
int* arr = new int[n];
```

- n can be taken at run time through **cin >> n;**
- A dynamic array is initialized at run time, on the *heap*, hence the name.
- Dynamic arrays have to be explicitly deallocated using the 'delete' keyword.
- delete[ ] arr; // deallocate memory
- **Static array →** fixed size, stack memory, auto-deleted. **Dynamic array →** flexible size, heap memory, must be explicitly deleted.




```
int* arr = new int[n];
```

arr stores the **address** to or **points to the starting location** of an array of integers of size n

Dynamic Array in C+

```cpp
#include <iostream>
using namespace std;

int main() {

    // Dynamic array (on the heap)
    int* dynamicArr = new int[3];
    dynamicArr[0] = 100;
    dynamicArr[1] = 200;
    dynamicArr[2] = 300;

    cout << "\nDynamic array addresses:\n";
    for (int i = 0; i < 3; i++) {
        cout << "&dynamicArr[" << i << "] = " << &dynamicArr[i]
             << " (same as "<< dynamicArr + i << ")" << endl;

        cout << "Values of dynamic array at " << i << dynamicArr[i] <<endl;
    }

    delete[] dynamicArr; // free memory
    return 0;
}
```

+

Exploration Exercise:

What is the output of the first cout?

What is the output of second cout?

What does the delete keyword do?

Note the syntax!!
Delete []
name_of_array

- Note dynamicArr[i] = *(dynamicArr +i)

More Examples:
Using an array name as a pointer (Ok)

```cpp
#include <iostream>
using namespace std;

int main() {
    int intarray[5] = {31, 54, 77, 52, 93};

    for (int j = 0; j < 5; j++) {
        cout << *(intarray + j) << endl;  // pointer arithmetic
    }

    return 0;
}
```

- Can we use `*intarray++`?
- Why or why not?

```cpp
#include <iostream>
using namespace std;

int main() {
    int intarray[5] = {31, 54, 77, 52, 93};
    int* p = intarray;   // pointer to first element

    for (int j = 0; j < 5; j++) {
        cout << *p << endl;
        p++;   // move to next element
    }

    return 0;
}
```

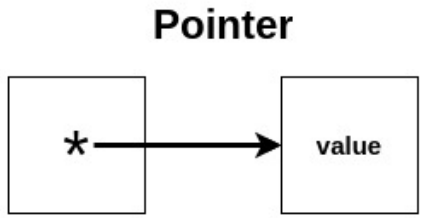
- Note that we can update the value of any pointer
- Difference between p++ and p+j is the value of pointer itself updates in

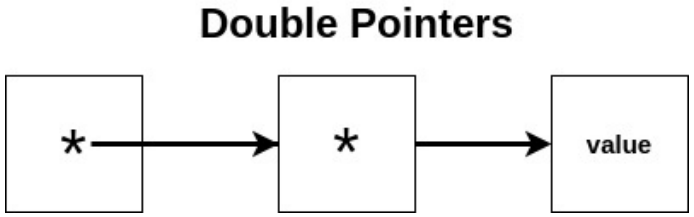
Double pointers

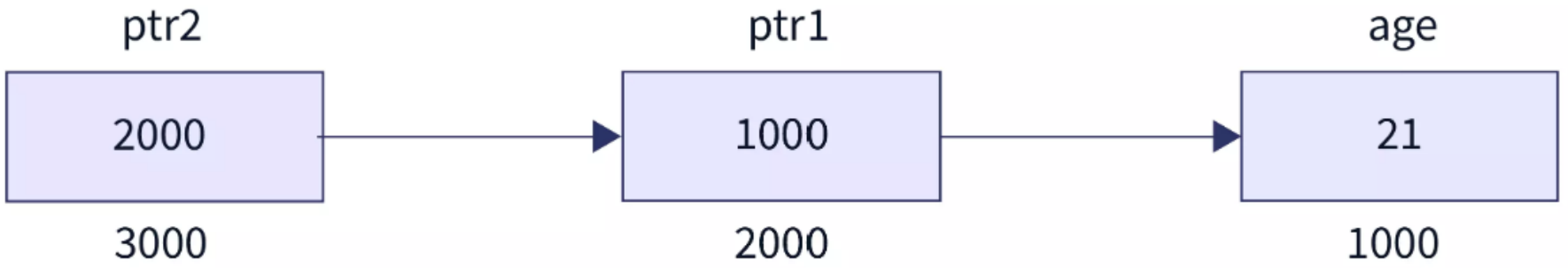
A pointer to a pointer?

Double Pointers

- Imagine a pointer that points to another pointer. That's what we call a double pointer.
- int** is a double pointer that store the address of another pointer.
- In C++, double pointers are used, among other things, to refer to 2-D arrays or linked lists of dynamic size
- int *ptr1 = &age; int **ptr2 = &ptr1;







Double Pointer Example!

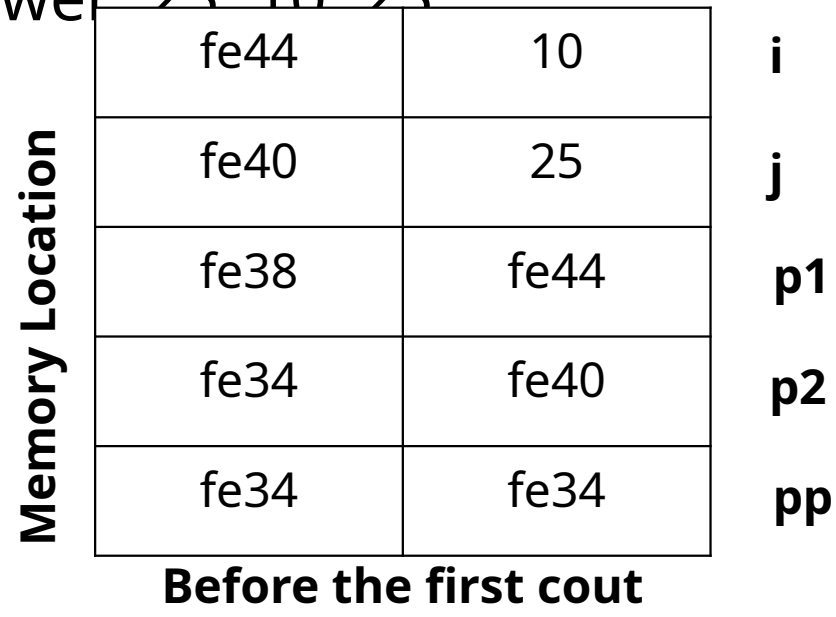
```
int main(){
    int i = 10;
    int j = 25;
    int *p1 = &i;
    int *p2 = &j;
    int **pp = &p2;
    cout << **pp << endl;
    pp = &p1;
    cout << **pp << endl;
    p1 = &j;
    cout << *p1 << endl;
}
```

- What are the 3 outputs?
- Can you visualize what values **i, j, p1 and p2** would hold after each assignment?

Double Pointer Example!

```
int main(){
    int i = 10;
    int j = 25;
    int *p1 = &i;
    int *p2 = &j;
    int **pp = &p2;
    cout << **pp << endl;
    pp = &p1;
    cout << **pp << endl;
    p1 = &j;
    cout << *p1 << endl;
}
```

- What are the 3 outputs?
- Can you visualize what values **i, j, p1 and p2** would hold after each assignment?
- Answer: 25, 10, 25



Example of Double Pointers with Dynamic 2D Array

Row = 3, Col = 4

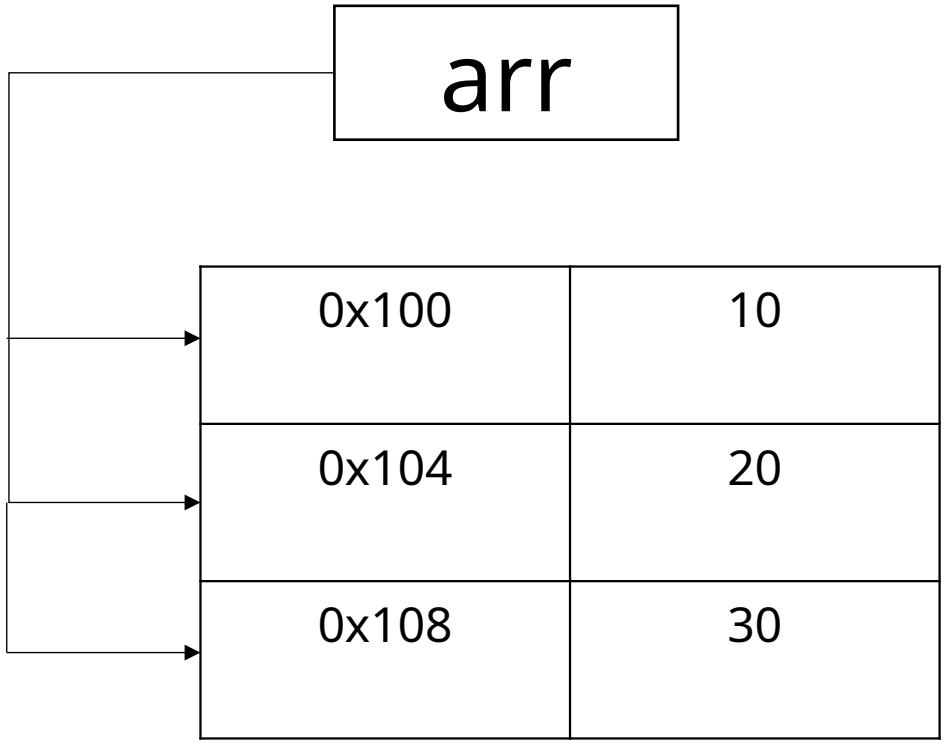
```
int **arr = new int*[n];
```

- Creates an array on n row pointers pointing to the beginning of a row.

- Visually: arr → [row0_ptr, row1_ptr, … row(n-1)_ptr].

Example of Double Pointers with Dynamic 2D Array

```
int **arr = new int*[n];
```



Code Example!

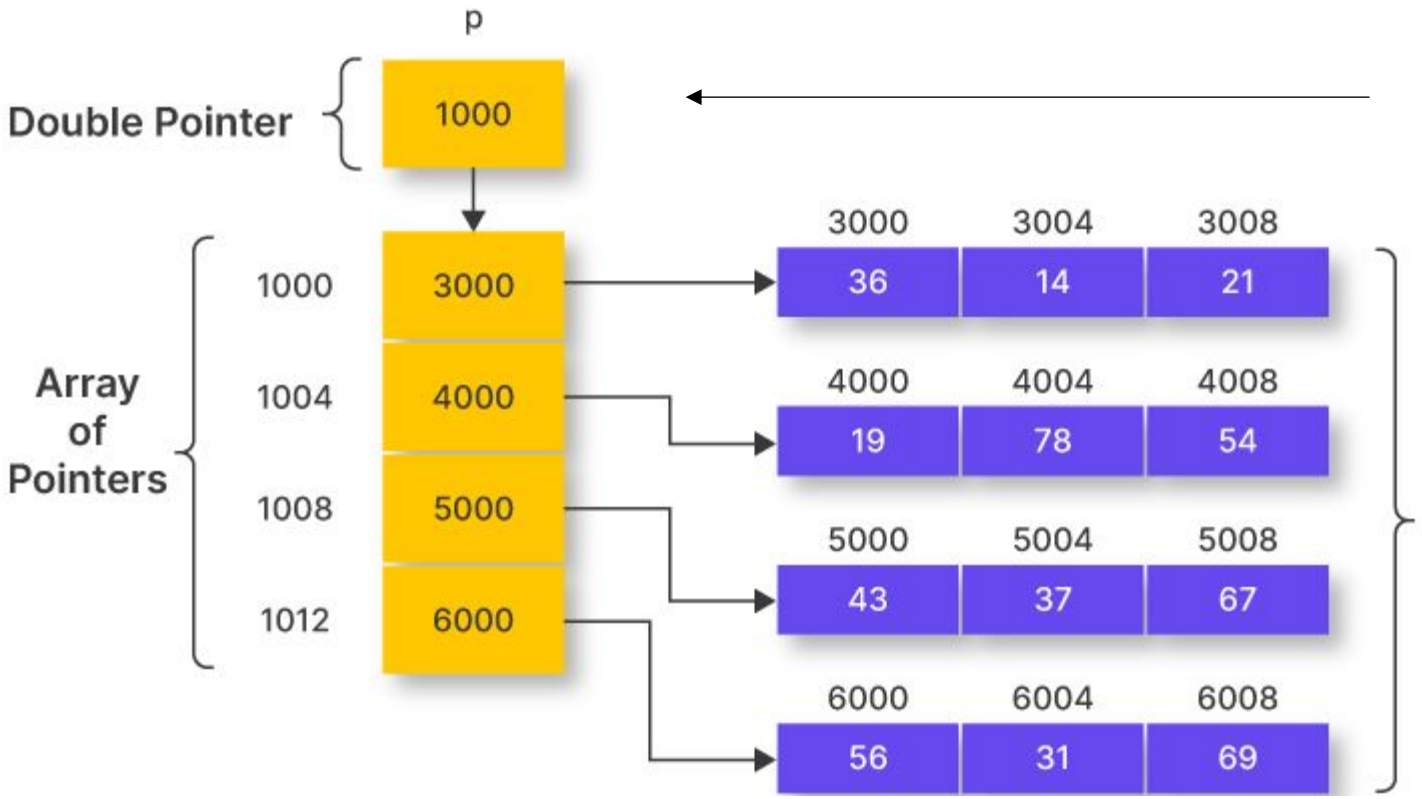
```
int rows, cols;
cout << "Enter number of rows: ";
cin >> rows;
cout << "Enter number of cols: ";
cin >> cols;

// Declare a double pointer and allocate memory for row pointers
int **arr = new int*[rows];

// Allocate memory for each row (columns)
for (int i = 0; i < rows; i++) {
    arr[i] = new int[cols];
}
```

- Loop goes through each row pointer.
- new int[cols] allocates space for m integers in each row.
- Altogether, this builds an rows × cols grid in memory, accessible as arr[i][j].
- Note: arr[i] is same as *(arr + i)

Do the usual stuff (but delete at the end!!)

```
// Fill the 2D array with sample values
cout << "\nFilling array with values i + j...\n";
for (int i = 0; i < rows; i++) {
    for (int j = 0; j < cols; j++) {
        arr[i][j] = i+j;  // sum
    }
}

// Print the 2D array
cout << "\n2D Array:\n";
for (int i = 0; i < rows; i++) {
    for (int j = 0; j < cols; j++) {
        cout << arr[i][j] << " ";
    }
    cout << endl;
}
```

```
// Free the allocated memory (important!)
for (int i = 0; i < rows; i++) {
    delete[] arr[i];
}
delete[] arr;
```



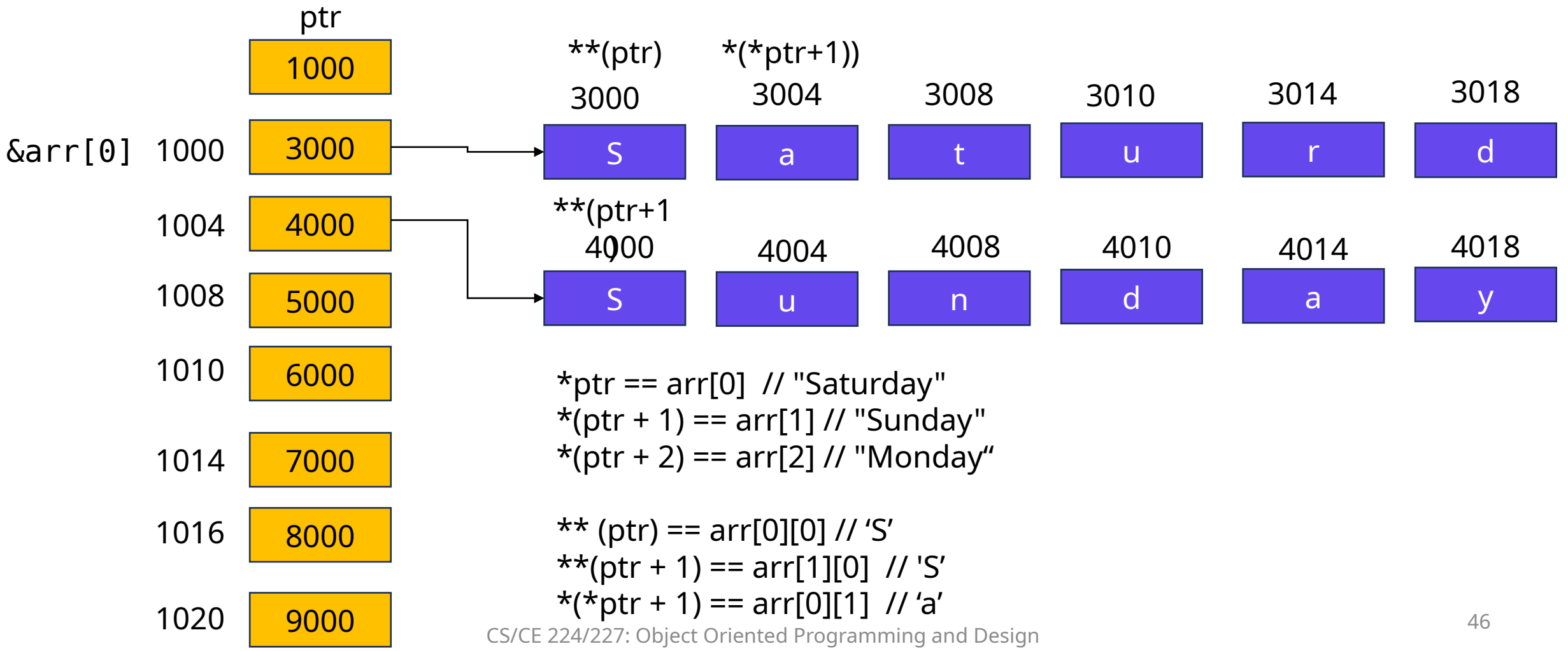
int **arr = new int*[n];
// starting memory address to an array of pointer locations in short **create a 1D array of pointers**

It is a double pointer so it the **values** stored in this array **are not integers** but **address of row locations** of column arrays

Double Pointers and Strings

```cpp
int main() {
    // Step 1: Declare an array of strings (array of char pointers)
    const char* arr[] = {"Saturday", "Sunday", "Monday", "Tuesday", "Wednesday", "Thursday", "Friday"};

    // Step 2: Declare a pointer to pointer (double pointer) to hold the array of strings
    const char** ptr = arr;

    // Step 3: Access the strings using the double pointer
    cout << "First string: " << *ptr << endl;     // Prints "Saturday"
    cout << "Second string: " << *(ptr + 1) << endl;  // Prints "Sunday"
    cout << "Third string: " << *(ptr + 2) << endl;   // Prints "Monday"

    // Step 4: Access individual characters in the string using pointer arithmetic
    cout << "First character of the second string: " << **(ptr + 1) << endl;  // Prints 'S'
    cout << "Second character of the first string: " << *(*ptr + 1) << endl;  // Prints 'S'

    return 0;
}
```

Visual explanation



*ptr == arr[0] // "Saturday"
*(ptr + 1) == arr[1] // "Sunday"
*(ptr + 2) == arr[2] // "Monday“

** (ptr) == arr[0][0] // ‘S’
**(ptr + 1) == arr[1][0] // 'S’
*(*ptr + 1) == arr[0][1] // ‘a’

- Answer:
- *ptr == arr[0] // "Saturday"
- *(ptr + 1) == arr[1] // "Sunday"
- *(ptr + 2) == arr[2] // "Monday“
- **(ptr + 1) == arr[1][0] // 'S’
- *(*ptr + 1) == arr[0][1] // ‘a’
  - (*ptr+1) points to a in Saturday
  - re-referencing it again gives a

```
int main() {
    // Step 1: Declare an array of strings (array of char pointers)
    const char* arr[] = {"Saturday", "Sunday", "Monday", "Tuesday", "Wednesday", "Thursday", "Friday"};

    // Step 2: Declare a pointer to pointer (double pointer) to hold the array of strings
    const char** ptr = arr;

    // Step 3: Access the strings using the double pointer
    cout << "First string: " << *ptr << endl;     // Prints "Saturday"
    cout << "Second string: " << *(ptr + 1) << endl;  // Prints "Sunday"
    cout << "Third string: " << *(ptr + 2) << endl;   // Prints "Monday"

    // Step 4: Access individual characters in the string using pointer arithmetic
    cout << "First character of the second string: " << **(ptr + 1) << endl;  // Prints 'S'
    cout << "Second character of the first string: " << *(*ptr + 1) << endl;  // Prints 'S'

    return 0;
}
```

Arrays, Pointers and Functions

- You have already seen how pointers can be used to pass addresses to individual variables.
- You may also pass a pointer to an array.
- However, it is recommended to pass the size of the array also as a parameter.
- Otherwise, this might lead to array decay – Explore – what is *array decay*?
- Note that arrays may also be passed by reference.
- Some examples follow.

```cpp
int main() {
    // Declare and initialize an array
    int arr[5] = {1, 2, 3, 4, 5};

    cout << "Before addOne: ";
    for (int i = 0; i < 5; ++i) {
        cout << arr[i] << " ";
    }
    cout << endl;

    // Call the function, passing the array by reference
    addOne(arr);

    // Print the array after modification
    cout << "After addOne: ";
    for (int i = 0; i < 5; ++i) {
        cout << arr[i] << " ";
    }
    cout << endl;

    return 0;
}
```

Passing an Array by Reference and by Pointer

```cpp
// Function to add 1 to each element
void addOne(int (&arr)[5]) {  // Array
    for (int i = 0; i < 5; ++i) {
        arr[i] += 1;
    }
}
```

```cpp
void addOne(int* arr, int size) {
    for(int i = 0; i < size; i++) {
        *(arr + i) += 1;
    }
}
```